부록 · LangChain4j · Spring AI

Spring AI vs LangChain4j

Spring Boot Java + Kotlin + Ollama Gemma3 실전 비교

1. 한눈에 보는 구조

```text
Spring AI: Spring Boot → Spring AI ChatModel → Ollama → Gemma3
LangChain4j: Spring Boot → LangChain4j ChatModel/AiService → Ollama → Gemma3

핵심: 둘 다 Spring Boot Java/Kotlin에서 사용할 수 있습니다. Spring AI는 Spring 생태계와의 자연스러운 통합이 강점이고, LangChain4j는 AiService·RAG·Tools·Agent 같은 LLM 애플리케이션 구성이 편리합니다.
```

2. Spring AI - 설정

```kotlin
dependencies {
    implementation("org.springframework.boot:spring-boot-starter-web")
    implementation("org.springframework.ai:spring-ai-starter-model-ollama")
}
```

```properties
spring.ai.ollama.base-url=http://localhost:11434
spring.ai.ollama.chat.model=gemma3
spring.ai.ollama.chat.temperature=0.2
```

3. Spring AI - Java

```java
@RestController
@RequestMapping("/api/ai")
public class SpringAiController {

    private final ChatModel chatModel;

    public SpringAiController(ChatModel chatModel) {
        this.chatModel = chatModel;
    }

    @GetMapping("/spring-ai")
    public Map<String, String> chat(@RequestParam String message) {
        String answer = chatModel.call(message);
        return Map.of("answer", answer);
    }
}
```

4. Spring AI - Kotlin

```kotlin
@RestController
@RequestMapping("/api/ai")
class SpringAiController(
    private val chatModel: ChatModel
) {
    @GetMapping("/spring-ai")
    fun chat(@RequestParam message: String): Map<String, String> {
        val answer = chatModel.call(message)
        return mapOf("answer" to answer)
    }
}
```

5. LangChain4j - 기본 구조

LangChain4j에서는 ChatModel을 직접 호출할 수도 있고,

AiService 인터페이스를 만들어 업무 의미가 드러나는 AI 서비스를 구성할 수도 있습니다.

Spring Boot → XrpAssistant → LangChain4j → Ollama → Gemma3

6. LangChain4j - Java ChatModel

```java
@RestController
@RequestMapping("/api/ai")
public class LangChainController {

    private final ChatModel chatModel;

    public LangChainController(ChatModel chatModel) {
        this.chatModel = chatModel;
    }

    @GetMapping("/langchain4j")
    public Map<String, String> chat(@RequestParam String message) {
        String answer = chatModel.chat(message);
        return Map.of("answer", answer);
    }
}
```

7. LangChain4j - Kotlin ChatModel

```kotlin
@RestController
@RequestMapping("/api/ai")
class LangChainController(
    private val chatModel: ChatModel
) {
    @GetMapping("/langchain4j")
    fun chat(@RequestParam message: String): Map<String, String> {
        val answer = chatModel.chat(message)
        return mapOf("answer" to answer)
    }
}
```

8. LangChain4j AiService - Java

```java
@AiService
public interface XrpAssistant {

    @SystemMessage("""
        당신은 XRP 기술적 분석 설명 AI입니다.
        제공된 기술지표를 이해하기 쉽게 설명하세요.
        확정적인 투자 조언은 하지 마세요.
        """)
    String analyze(String message);
}
```

```java
@RestController
@RequestMapping("/api/xrp")
public class XrpController {

    private final XrpAssistant assistant;

    public XrpController(XrpAssistant assistant) {
        this.assistant = assistant;
    }

    @GetMapping("/analysis")
    public String analysis() {
        return assistant.analyze("""
            XRP
            MA5=4200
            MA20=4100
            RSI14=63
            MACD Histogram=15
            분석해주세요.
            """);
    }
}
```

9. LangChain4j AiService - Kotlin

```kotlin
@AiService
interface XrpAssistant {

    @SystemMessage(
        """
        당신은 XRP 기술적 분석 설명 AI입니다.
        제공된 기술지표를 이해하기 쉽게 설명하세요.
        확정적인 투자 조언은 하지 마세요.
        """
    )
    fun analyze(message: String): String
}
```

```kotlin
@RestController
@RequestMapping("/api/xrp")
class XrpController(
    private val assistant: XrpAssistant
) {
    @GetMapping("/analysis")
    fun analysis(): String =
        assistant.analyze(
            """
            XRP
            MA5=4200
            MA20=4100
            RSI14=63
            MACD Histogram=15
            분석해주세요.
            """.trimIndent()
        )
}
```

10. XRP 분석 프로젝트에 적용

```text
UpbitService
    ↓
IndicatorService
    ↓
IndicatorDto
    ↓
SignalService
    ↓
SignalDto
    ↓
[선택 A] Spring AI ChatModel
또는
[선택 B] LangChain4j XrpAssistant
    ↓
Ollama (http://localhost:11434)
    ↓
Gemma3
    ↓
자연어 분석
```

11. Spring AI vs LangChain4j 특징 비교표

항목	Spring AI	LangChain4j
주요 방향	Spring 중심 AI 통합	Java 중심 LLM 애플리케이션
Spring Boot 친화성	매우 높음	높음
Java	좋음	좋음
Kotlin	좋음	좋음
Ollama/Gemma3	지원	지원
기본 모델 추상화	ChatModel	ChatModel

항목	Spring AI	LangChain4j
간단 호출	call()	chat()
고수준 AI 서비스	Spring 방식 구성	AiService가 편리
Prompt	지원	지원
Streaming	지원	지원
Tool Calling	지원	지원
Embedding	지원	지원
Vector Store	지원	지원
RAG	지원	지원
Chat Memory	지원	강점
Agent/Tools 확장	가능	강점
초보자 기본 Chat	쉬움	쉬움
복잡한 AI 서비스	좋음	매우 편리
XRP 분석 프로젝트	적합	매우 적합

12. Java와 Kotlin 비교

항목	Java	Kotlin
생성자 주입	필드+생성자	주 생성자로 간결
Map 반환	Map.of()	mapOf()
멀티라인 문자열	text block	triple-quoted string
DTO	record	data class
Spring AI 기능	동일	동일
LangChain4j 기능	동일	동일
코드량	상대적으로 많음	상대적으로 적음

13. 추천 학습 순서

1) Spring Boot REST API
2) Spring AI ChatModel + Ollama/Gemma3
3) LangChain4j ChatModel
4) LangChain4j AiService
5) RAG
6) Chat Memory
7) Tool Calling
8) Agent
9) 기존 XRP IndicatorService/SignalService/MultiTimeframeService와 연결

현재 XRP 프로젝트는 LangChain4j 구조를 계속 유지해도 좋습니다. 학습 목적으로 동일 기능을 Spring AI와 LangChain4j 두 버전으로 구현하면 두 프레임워크의 차이를 가장 빨리 이해할 수 있습니다.
